

1. Introduction to Compilers

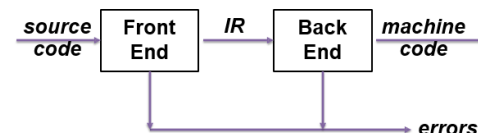
- **Definition:**
 - A compiler translates programs from a **source language** (e.g., FORTRAN, Pascal) to a **target language** (e.g., machine language or another programming language).
 - Reports errors in the source program during translation.
- **Variety:**
 - Thousands of source languages exist, including specialized and traditional programming languages.
 - Target languages vary widely, including machine languages and intermediate languages.
- **Types of Compilers:**
 - **Single-pass:** Processes the source code in one go.
 - **Multi-pass:** Processes the source code multiple times to refine the output.
 - **Load-and-go:** Executes the program immediately after compilation.
 - **Debugging:** Helps in identifying and resolving issues in the source code.
 - **Optimizing:** Improves the performance of the target code.



2. The Analysis-Synthesis Model of Compilation

- Compilation is divided into two parts:
 - **Analysis:** Breaks the source program into pieces and creates an intermediate representation.
 - **Synthesis:** Converts the intermediate representation into the target program.

The Analysis-Synthesis Model of Compilation:



Analysis Phase

1. **Linear Analysis (Lexical Analysis/Scanning):**
 - Reads characters from left to right and groups them into **tokens**.
 - Eliminates irrelevant characters, such as spaces.
 - Example:
 - Input: position := initial + rate * 60
 - Tokens: position, :=, initial, +, rate, *, 60.
2. **Hierarchical Analysis (Syntax Analysis/Parsing):**
 - Groups tokens into grammatical phrases and creates a **parse tree**.
 - Syntax trees are used to represent the hierarchical structure.
 - Example:
 - Expression: initial + rate * 60
 - rate * 60 is grouped before initial + due to operator precedence.
3. **Semantic Analysis:**
 - Ensures the program components are meaningful (e.g., type checking).
 - Example:
 - A binary arithmetic operator applied to an integer and real number may require type conversion (e.g., integer to real).

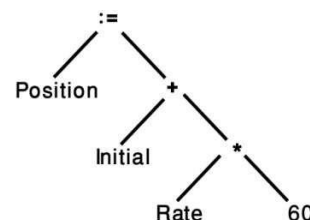
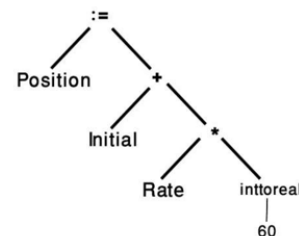


Fig 1.2. Syntax tree of position :=initial + rate * 60

Synthesis Phase

- Constructs the target program using the intermediate representation.
- Syntax trees guide the construction process.

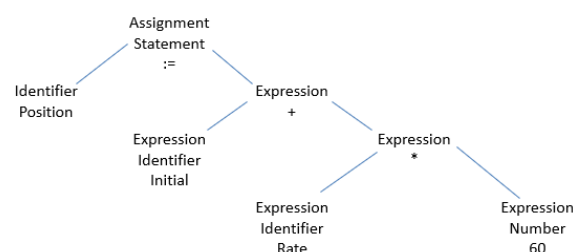
Semantic Analysis:



3. Compiler Phases

1. **Lexical Analysis:**
 - Converts characters into tokens and eliminates unnecessary spaces.
 - Outputs a stream of tokens.
2. **Syntax Analysis:**
 - Organizes tokens into a structured parse tree using grammar rules.
 - Example grammar rules:

Syntax Analysis or Hierarchical Analysis (Parsing):



- Any identifier or number is an expression.
- Expressions can combine using operators like +, *.

3. Semantic Analysis:

- Checks for semantic correctness, such as valid types and operator usage.
- Example: Ensures operands in $a + b$ are compatible types.

4. Intermediate Code Generation:

- Produces an intermediate representation, such as **three-address code**.
- Example:
 - Input: $id1 := id2 + id3 * 60$
 - Output:
 - $temp1 := id3 * 60$
 - $temp2 := id2 + temp1$
 - $id1 := temp2$

5. Code Optimization:

- Improves intermediate code for better performance.
- Example: Merge instructions:
 - Original:
 - $temp1 := id3 * 60.0$
 - $id1 := id2 + temp1$
 - Optimized:
 - $id1 := id2 + (id3 * 60.0)$

6. Code Generation:

- Converts intermediate code to machine code.
- Assigns memory locations or registers to variables.
- Example machine code:
 - `MOVF id3, r2`
 - `MULF #60.0, r2`
 - `MOVF id2, r1`
 - `ADDF r2, r1`
 - `MOVF r1, id1`

4. Error Handling

- Errors can occur during lexical, syntax, or semantic analysis.
- The compiler attempts to handle errors and continues to detect further issues without stopping after the first error.
- Examples:
 - Lexical errors: Invalid characters.
 - Syntax errors: Violations of grammar rules.
 - Semantic errors: Type mismatches.

5. Additional Language Processing Components

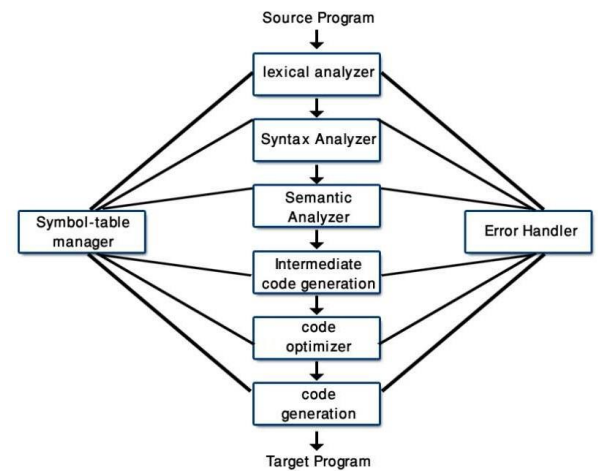
1. Preprocessors:

- Perform tasks like macro processing and file inclusion.
- Example: The C preprocessor includes header files.

2. Assemblers:

- Translate assembly code (mnemonic form) to machine code.
- Example:
 - Assembly:
 - `MOV a, R1`
 - `ADD #2, R1`
 - `MOV R1, b`
 - Equivalent in machine code:
 - Computes $b := a + 2$.

3. Loaders/Linkers:



- Link multiple relocatable machine code files into a single executable.
- Resolve addresses for memory allocation.

6. Compiler Architecture

- **Front-End:**
 - Source-language dependent tasks like lexical and syntax analysis, symbol table creation, and semantic analysis.
 - Generates intermediate code.
 - Handles front-end error reporting.
- **Back-End:**
 - Target-language dependent tasks like code optimization and code generation.
 - Uses intermediate code as input.

7. Compiler-Construction Tools

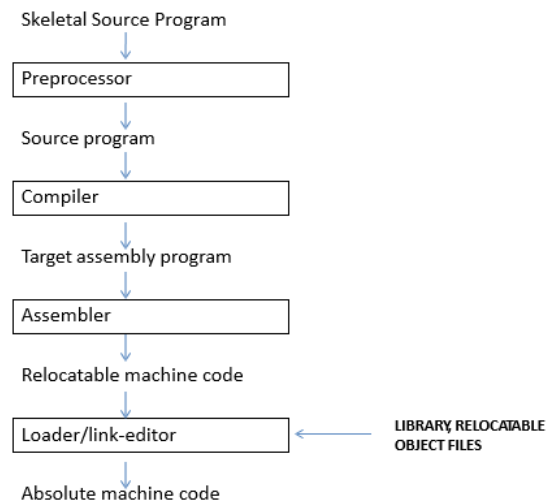
1. **Parser Generators:**
 - Create syntax analyzers from context-free grammar rules.
2. **Scanner Generators:**
 - Generate lexical analyzers based on regular expressions.
3. **Syntax-Directed Translation Engines:**
 - Traverse parse trees to generate intermediate code.
4. **Automatic Code Generators:**
 - Translate intermediate code to machine code for the target language.
5. **Data-Flow Engines:**
 - Analyze value transmission for optimization.

Language Processing System

A **Language Processing System** consists of several components that work together to convert high-level source code into machine-executable code. Here's an overview:

Stages of a Language Processing System

1. **Skeletal Source Program:**
 - The original program written by the programmer.
2. **Preprocessor:**
 - Processes the skeletal source code to prepare it for compilation.
 - Functions include:
 - **Macro Processing:** Expands macros (shortcuts for repetitive code).
 - **File Inclusion:** Inserts code from header files.
 - **Language Extensions:** Adds new capabilities (e.g., database access).
3. **Compiler:**
 - Converts the preprocessed source program into assembly language or intermediate code.
 - Major tasks:
 - Lexical, syntax, and semantic analysis.
 - Intermediate code generation and optimization.
4. **Assembler:**
 - Converts assembly language (a mnemonic version of machine language) into relocatable machine code.
 - Example:
 - Assembly: MOV a, R1
 - Machine Code: Binary sequence.
5. **Loader and Linker:**
 - **Loader:**
 - Loads the relocatable machine code into memory.
 - Adjusts addresses for relocatability.



1.3. A LANGUAGE-PROCESSING SYSTEM

- **Linker:**
 - Combines multiple relocatable code files into a single executable program.
- 6. **Absolute Machine Code:**
 - The final executable code, ready to run on the machine.

Flow in a Language Processing System

1. **Source Program → Preprocessor**
Produces preprocessed code.
2. **Preprocessed Code → Compiler**
Produces assembly code or intermediate representation.
3. **Assembly Code → Assembler**
Produces relocatable machine code.
4. **Relocatable Machine Code → Loader/Linker**
Produces absolute machine code.

Example: Processing in C Language

1. **Source Code:**
2. `#include <stdio.h>`
3. `int main() {`
4. `printf("Hello, World!\n");`
5. `return 0;`
6. `}`
7. **Preprocessor:** Expands `#include <stdio.h>` with the actual code in `stdio.h`.
8. **Compiler:** Translates the code into assembly language.
9. **Assembler:** Converts assembly to relocatable machine code.
10. **Linker/Loader:** Links with standard libraries (e.g., `printf`) to create the final executable.

Analysis and Synthesis Model of a Compiler

Focusing on **Lexical Analysis**, **Syntax Analysis**, and **Semantic Analysis**.

1. Analysis Phase

The **Analysis Phase** is responsible for processing the source code and breaking it into manageable components, ensuring it adheres to language rules, and preparing it for further processing. This phase includes **Lexical Analysis**, **Syntax Analysis**, and **Semantic Analysis**.

1.1 Lexical Analysis

Role:

- The first step in the analysis phase, also known as **tokenization** or **scanning**.
- Converts a sequence of characters from the source program into **tokens**, which are the smallest units of meaningful code.

Process:

1. Reads the source code character by character from left to right.
2. Groups characters into tokens, such as keywords, operators, identifiers, or literals.
3. Eliminates whitespace and comments, which are not meaningful for further processing.

Output:

- A stream of tokens.

Input: `position := initial + rate * 60;`

Output (Tokens):

- `position` (Identifier)
- `:=` (Assignment Operator)
- `initial` (Identifier)
- `+` (Operator)
- `rate` (Identifier)
- `*` (Operator)
- `60` (Literal)

1.2 Syntax Analysis

Role:

- Also called **parsing**, this phase ensures that the stream of tokens follows the **grammar rules** of the programming language.
- Constructs a **parse tree** or **syntax tree** to represent the program structure hierarchically.

Process:

1. Takes the token stream from the lexical analysis.
2. Applies the grammar rules of the language.
3. Groups tokens into higher-level constructs like expressions, statements, and functions.

Output:

- A **parse tree**, showing the hierarchical structure of the source code.

Expression: position := initial + rate * 60;

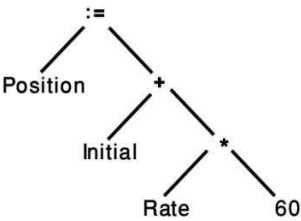


Fig 1.2. Syntax tree of position :=initial + rate * 60

1.3 Semantic Analysis

Role:

- Ensures that the program components are **meaningful** and conform to **semantic rules** (e.g., type checking, variable declarations).
- Checks the relationships between components.

Process:

1. Analyzes the parse tree or abstract syntax tree (AST) from the syntax analysis.
2. Ensures type compatibility, variable declarations, and proper usage of constructs.

Key Tasks:

- **Type Checking:** Ensures operations are applied to compatible data types (e.g., integers and floats).
- **Scope Resolution:** Ensures identifiers are declared before use.
- **Implicit Type Conversions:** Converts data types where necessary (e.g., integer to float).

Output:

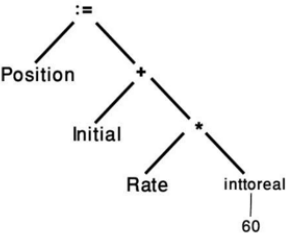
- An **annotated parse tree** or AST with type information.

Expression: position := initial + rate * 60;

Semantic Rules:

1. Ensure position, initial, and rate are declared and within scope.
2. Check type compatibility:
 - If rate is a float and 60 is an integer, the integer may need conversion to float.
3. Annotated Tree:
4. Assignment
5. |— Identifier (position) [type: float]
6. |— :=
7. |— Expression
8. |— |— Identifier (initial) [type: float]
9. |— |— +
10. |— |— Expression
11. |— |— |— Identifier (rate) [type: float]
12. |— |— |— *
13. |— |— |— Literal (60) [converted to float]

Semantic Analysis:



Key Comparison

Phase	Focus	Output
Lexical Analysis	Tokenizing source code	Tokens (e.g., position, :=, rate)
Syntax Analysis	Constructing hierarchical structure	Parse tree (represents grammar structure)
Semantic Analysis	Enforcing semantic correctness	Annotated tree with semantic information

Diagram of the Analysis Process

Source Code → Lexical Analysis → Tokens → Syntax Analysis → Parse Tree → Semantic Analysis → Annotated Parse Tree

Introduction

- **Lexical Analysis:** The first phase of a compiler, often called **tokenization**.
- **Role:** Reads characters from the source program and groups them into **lexemes** (meaningful sequences).
- **Output:** Tokens in the format <token-name, attribute-value>.
 - Example: position becomes <id, 1> where id represents "identifier" and 1 points to the symbol table.

Key Terms

1. **Lexeme:** Sequence of characters in the source program matching a token's pattern.
 - Example: For the code position = initial + rate * 60:
 - position → <id, 1>
 - initial → <id, 2>
 - + → <+, >
 - 60 → <60, >
2. **Token:** A pair containing a token name and an optional attribute value.
 - Example:
 - Identifiers: {i, sum}
 - Operators: {+, -}
 - Reserved words: {for, int}
3. **Pattern:** Rule describing the set of lexemes a token can represent.
 - For keywords, the pattern is the exact sequence of characters forming the keyword.
 - For identifiers, it matches strings that begin with a letter and are followed by letters/digits.

Role of the Lexical Analyzer

- **Primary Role:** Produce tokens for the syntax analyzer.
- **Interacts with the Symbol Table:**
 - Inserts identifiers and retrieves attributes for semantic analysis.
- **Additional Tasks:**
 - Removes comments and whitespace.
 - Correlates error messages with source lines.
 - Expands macros (if applicable).

Subtasks

1. **Scanning:** Deletes comments and compresses consecutive whitespace.
2. **Tokenization:** Groups characters into tokens.

Lexical Analysis vs. Parsing

Reasons for Separating Lexical and Syntax Analysis:

1. **Simplified Design:**
 - Lexical analysis focuses on tokenization.
 - Parsing handles syntax structure.
2. **Efficiency:**
 - Specialized techniques optimize lexical tasks.
3. **Portability:**
 - Device-specific input handling is confined to the lexical analyzer.

Handling Lexical Errors

1. **Types of Errors:**
 - **Exceeding Length:** Lexemes longer than language specifications (e.g., numeric constants exceeding precision).
 - **Illegal Characters:** Unsupported symbols like ~ or →.
 - **Unterminated Strings/Comments.**
2. **Error Recovery Strategies:**
 - **Panic Mode:** Delete characters until a valid token is found.
 - **Other Actions:**
 - Insert missing characters.
 - Replace incorrect characters.
 - Transpose adjacent characters (e.g., fi → if).

Input Buffering

1. **Challenges:**
 - Sometimes, multiple characters need to be read to confirm a token.
 - Example: Differentiating between -, --, and -> in C.
2. **Two-Buffer Scheme:**
 - Two alternating buffers handle large look-ahead safely.
 - Uses **sentinels** (special EOF markers) to avoid frequent checks for the end of buffers.
3. **Pointers:**
 - `lexemeBegin`: Marks the start of the current lexeme.
 - `forward`: Scans ahead to find the token's end.

Specification of Tokens

1. **Regular Expressions:**
 - Used to define patterns for tokens.
 - Effective for tokens like identifiers, constants, and reserved words.
2. **Examples:**
 - **C++ Identifiers:**
 - $ID \rightarrow \text{letter} (\text{letter} \mid \text{digit})^*$
 - $\text{letter} \rightarrow [A-Za-z_]$
 - $\text{digit} \rightarrow [0-9]$
 - **Unsigned Numbers:**
 - $\text{number} \rightarrow \text{digit}+ (\cdot \text{digit}+)? (E[+-]? \text{digit}+)?$

Recognition of Tokens

1. **Transition Diagrams:**
 - Graphical representation of token recognition using finite state automata.
 - States represent decisions; transitions define valid inputs.
2. **Examples:**
 - **Operators:** Diagrams to handle +, -, >=, etc.
 - **Identifiers:** $\text{letter} (\text{letter} \mid \text{digit})^*$.
 - **Whitespace:** $(\text{blank} \mid \text{tab} \mid \text{newline})^+$.
3. **Longest Match Rule:**
 - Ensures the longest valid token is selected (e.g., `next` as an identifier, not a part of another token).

Syntax Analysis - Notes

Syntax Analysis

- **Input:** Stream of tokens from the lexical analyzer.
- **Output:** Parse tree or Abstract Syntax Tree (AST).
- **Side Effect:** Populates the symbol table.
- **Errors:** Syntax errors occur when input does not conform to grammar.

Syntax Analyzer (Parser)

- Creates a syntactic structure (parse tree) from input tokens.
- Uses **Context-Free Grammar (CFG)** to describe the syntax of programming languages.

Ambiguity in Grammar

- **Ambiguous Grammar:** Produces more than one parse tree for the same string.
- Example:
 - $\text{expr} \rightarrow \text{expr} + \text{expr} \mid \text{expr} * \text{expr} \mid \text{digit}$
- For the string `9 + 5 * 2`:
 - Parse 1: $(9 + (5 * 2))$
 - Parse 2: $((9 + 5) * 2)$

Drawbacks of Ambiguity:

- Leads to ambiguous semantics.
- Increases parsing complexity.

- Affects other compilation phases.

Removing Ambiguity

1. No general algorithm exists to remove ambiguity or determine if a CFG is ambiguous.
2. **Techniques:**
 - Respect operator precedence (e.g., * before +).
 - Enforce associativity:
 - Left-associative: $(3 + 4) + 5$
 - Right-associative: $3 + (4 + 5)$
3. Rewrite ambiguous grammar to incorporate precedence and associativity rules.
Example (Unambiguous Grammar):
4. $\text{expr} \rightarrow \text{expr} + \text{term} \mid \text{term}$
5. $\text{term} \rightarrow \text{term} * \text{factor} \mid \text{factor}$
6. $\text{factor} \rightarrow \text{digit} \mid (\text{expr})$

Feature	Compiler	Interpreter
Translation	Converts the entire source code into machine code at once.	Translates and executes the source code line by line.
Output	Produces an executable file.	Does not produce an executable; runs code directly.
Execution Speed	Faster execution (compiled code is optimized and pre-translated).	Slower (each line is translated and executed during runtime).
Errors	Displays all errors after compilation.	Stops and reports errors as they occur.
Examples	C, C++, Rust	Python, JavaScript, PHP

lexemes

1. Keywords

Lexeme	Pattern	Token	Attribute Value
if, else	Fixed sequence	<keyword, lexeme>	Literal keyword itself (if, else)
for, while	Fixed sequence	<keyword, lexeme>	Literal keyword itself (for, while)
int, float	Fixed sequence	<keyword, lexeme>	Literal keyword itself (int, float)

2. Identifiers

Lexeme	Pattern	Token	Attribute Value
varName, _x1	$[A-Za-z_][A-Za-z0-9_]^*$	<id, entry>	Pointer to symbol table entry

3. Constants

Integer Constants:

Lexeme	Pattern	Token	Attribute Value
10, -3	$[+-]?[0-9]^+$	<int_const, value>	Integer value (10, -3)

Floating-Point Constants:

Lexeme	Pattern	Token	Attribute Value
3.14	$[+-]?[0-9]^*\.[0-9]^+$	<float_const, value>	Floating-point value (3.14)
2.1e+5	$[+-]?[0-9]^*\.[0-9]^+[eE][+-]?[0-9]^+$	<float_const, value>	Scientific notation value (2.1e+5)

Character Constants:

Lexeme	Pattern	Token	Attribute Value
'a'	$['^']$	<char_const, value>	Character value ('a')

String Constants:

Lexeme	Pattern	Token	Attribute Value
"hello"	"[^"]*"	<string_const, value>	String value ("hello")

4. Operators

Arithmetic Operators:

Lexeme	Pattern	Token	Attribute Value
+, -	Fixed sequence	<operator, lexeme>	Operator itself (+, -)

Relational Operators:

Lexeme	Pattern	Token	Attribute Value
<, >=	`<=	>=	==

Logical Operators:

Lexeme	Pattern	Token	Attribute Value
&&, `		`	Fixed sequence

5. Separators

Lexeme	Pattern	Token	Attribute Value
(,)	Fixed sequence	<separator, lexeme>	Separator itself
{, }	Fixed sequence	<separator, lexeme>	Separator itself
[,]	Fixed sequence	<separator, lexeme>	Separator itself
;, ,	Fixed sequence	<separator, lexeme>	Separator itself

6. Comments

Single-Line:

Lexeme	Pattern	Token	Attribute Value
// This is a comment	//.*	<comment, ignored>	Ignored in lexical analysis

Multi-Line:

Lexeme	Pattern	Token	Attribute Value
/* This is a comment */	/*.*?*/	<comment, ignored>	Ignored in lexical analysis

7. Whitespace

Lexeme	Pattern	Token	Attribute Value
, \t	[\t\n]+	<whitespace, ignored>	Ignored in lexical analysis

Example: Tokenizing a Code Snippet

Code:

int sum = 10 + 20;

Lexeme	Pattern	Token	Attribute Value
int	int	<keyword, int>	int
sum	[A-Za-z_][A-Za-z0-9_]*	<id, entry>	Symbol table pointer
=	=	<operator, =>	=
10	[0-9]+	<int_const, value>	10
+	+	<operator, +>	+
20	[0-9]+	<int_const, value>	20
;	;	<separator, ;>	;